

データベース演習

第14回：最終課題 ER図の作成

鈴木源吾

前回の復習

1. セッション分析：アクセス間隔を求める
 - LAGウィンドウ関数の利用
2. セッション分析：セッション識別
 - CASE式の利用

今回のトピック

1. 最終課題用のデータベース作成と課題の説明
2. ER図，ER図とテーブルとの対応の復習
3. 最終課題データベースのER図作成
4. 発展問題：データベースの拡張設計（問7）

1. 最終課題用のデータベース作成と課題の説明

最終課題用のデータベース作成

最終課題に使うデータベース ex_final は，これまでの回と同様にSQLiteファイルとして提供する

- 配布ノートブック（2012xxxx_DS_14.ipynb）の冒頭のセルを実行してダウンロードし，接続する
- Colabのランタイムが切り替わるたびにダウンロードし直すこと

正しくデータベースが作成されているかを，ノートブック上で以下を確認すること

- テーブルが14個あること
 - 確認するSQL文：SELECT count(*) FROM sqlite_master WHERE type='table';
- テーブル ordersのタプル数が830であること
 - 確認するSQL文：SELECT count(*) FROM orders;

最終課題で想定している状況

- 帳票からデータベースの設計を行いER図を作成する手法をボトムアップアプローチと呼んだ
- 今回は、既存のデータベースがあり、それを拡張・改造したいという状況とする
- しかし、残されたドキュメントが貧弱な場合があり、今回はデータベース仕様書が貧弱という状況である
- このようなケースは、実際のシステムでも、（不幸なことだが）起こりうる
- これもボトムアップアプローチの一種である

最終課題の概要

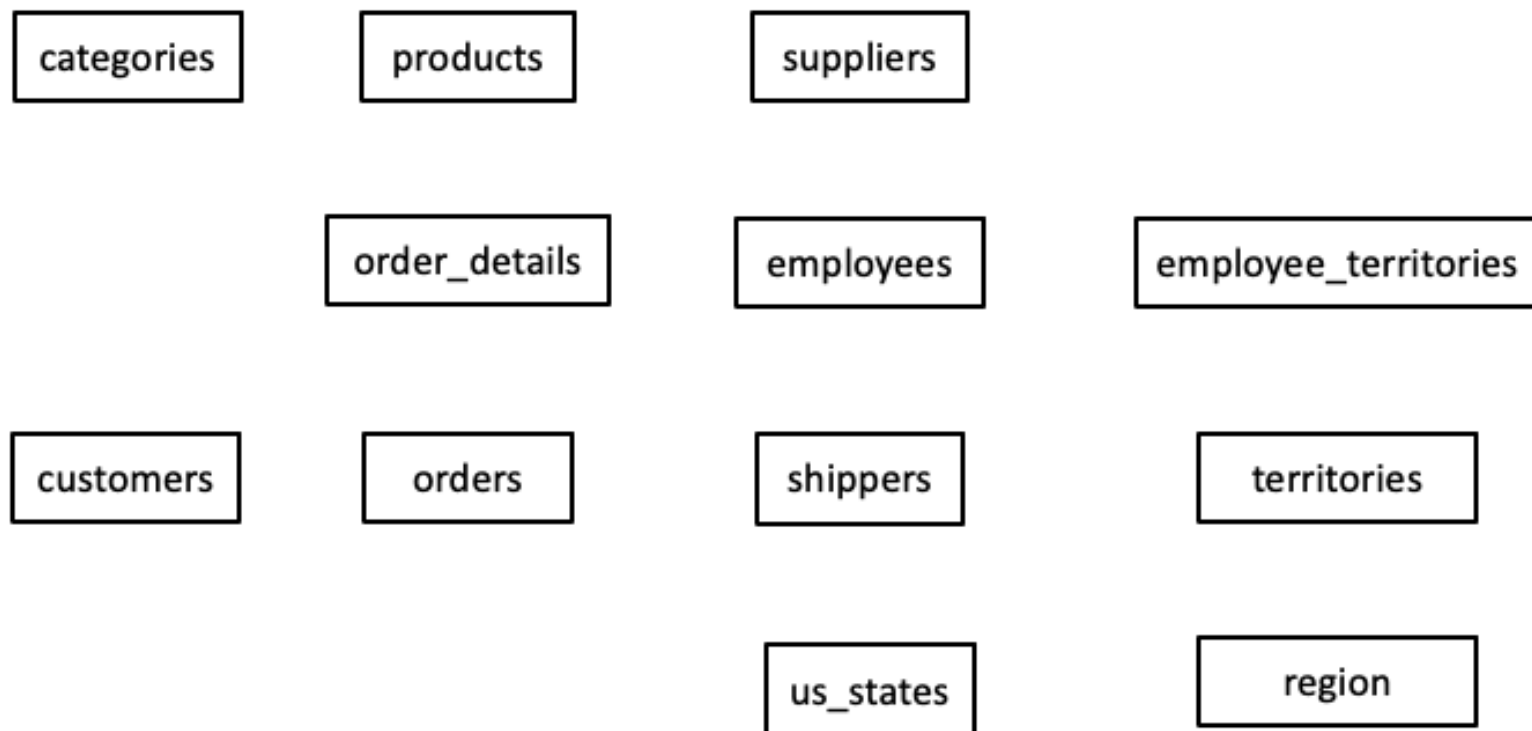
- ex_finalデータベースは，ネットで注文を受けた製品を発送するための情報を管理するデータベースである．このex_finalデータベースの改善を行う
 - 注文(orders)，顧客(customers)，製品(products)，注文を処理する従業員(employees)などの情報を管理している
 - テーブルのうち，customer_customer_demoとcustomer_demographicsテーブルは使用しない
- このデータベースを用い，最終課題では以下を行う
 - 設計の調査：データベースのデータや仕様書を調査しER図を作成
 - 拡張の設計（発展）：新しい要件に対するテーブル設計の提案
 - アプリケーション作成：データベースにアクセスするプログラム作成
 - データ分析：データベースのデータを分析するSQL文の作成

今回の演習の概要

- ex_finalデータベースに含まれるテーブルなどに関する仕様書を配布する
 - 学生ポータルにアップロードする
 - 仕様書は**閲覧用の参考資料**である（記入・提出には使わない）
- ex_finalのデータがどうなっているかを，データベース仕様書も参考にしながら，ノートブック上でSQL文を発行して調査する
 - メタデータ（主キーなど）もSQL（PRAGMA関数）で調査できる（後述）
- 調査結果はノートブックの記入表に記録し，それも踏まえてex_finalのER図を完成させる

データベースに含まれるテーブル

データベースには以下のテーブルが含まれる．これらのテーブルをエンティティ（実体）とみなし，関連の線（多重度を表す矢印とオプションリティを表す黒丸・白丸含む）を加筆してER図を作成する



注意：customer_customer_demoとcustomer_demographicsは使用しない

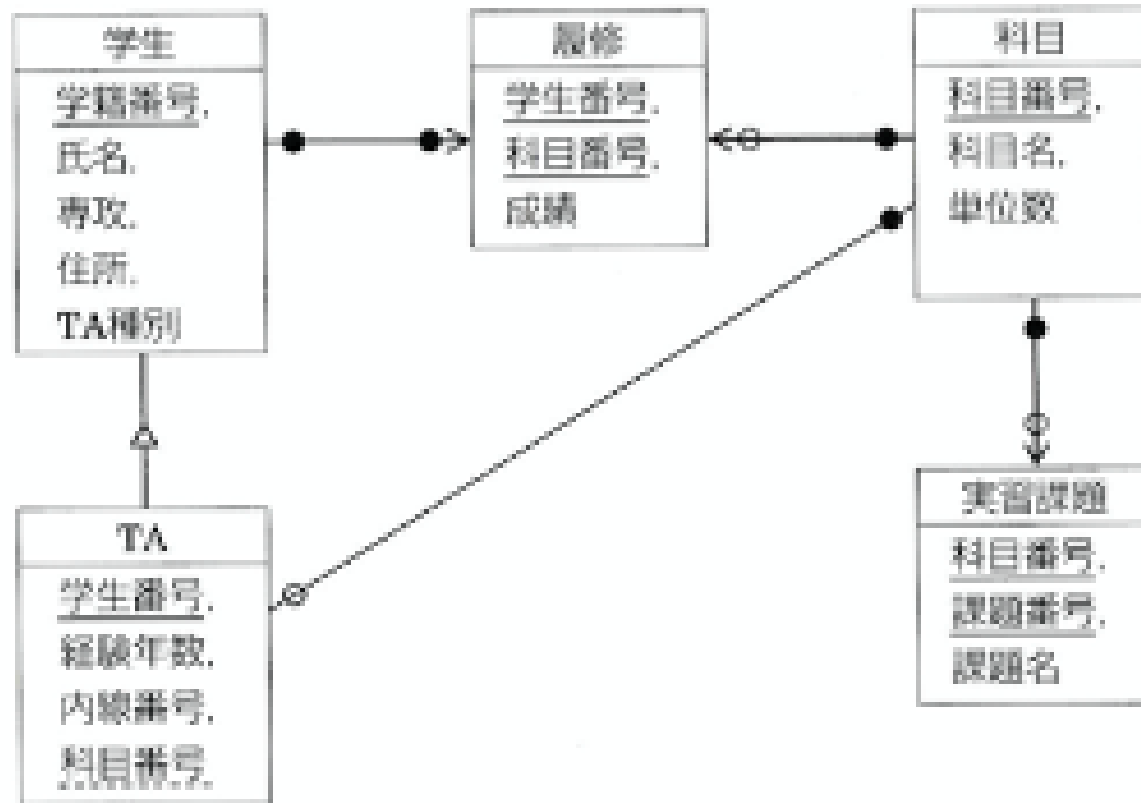
データベース仕様書：テーブルの構造とデータの制約

- データベース仕様書（PDF）は2部構成で、テーブルの構造（テーブルとカラム）と、データの制約に関する仕様が記載されている
 - **主キー・外部キー・関連についての記載はない**．これらを調査して、ER図の関連を作成するのが最終課題である
 - データの制約に関する仕様を参考に、オプションリティを決定する．ただし、これだけで決定できない箇所はデータを調べて決定する
 - 調査結果の記録・提出は配布ノートブック上で行う

2. ER図，ER図とテーブルとの対応の復習

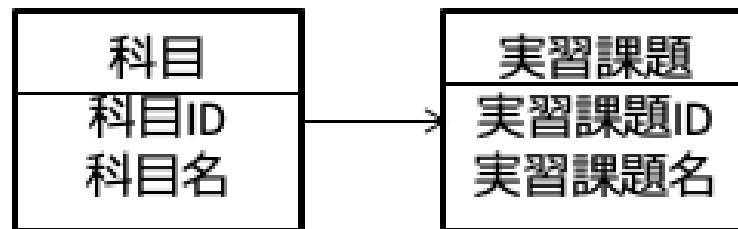
ER図

ER図は，実体（エンティティ）と関連によって，情報の構造を表す手法である．今回は，以下の情報処理試験の表記法を用いる

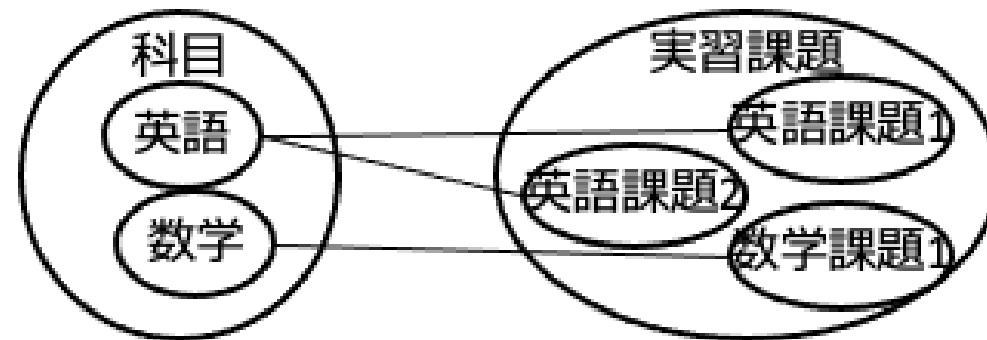


ER図：多重度

- 関連に対応する実体の個数に関するルールを多重度と呼ぶ
- ER図上では，多重度を関連を表す直線の矢印で表現する
- 複数ある側に矢印がつく



ER図



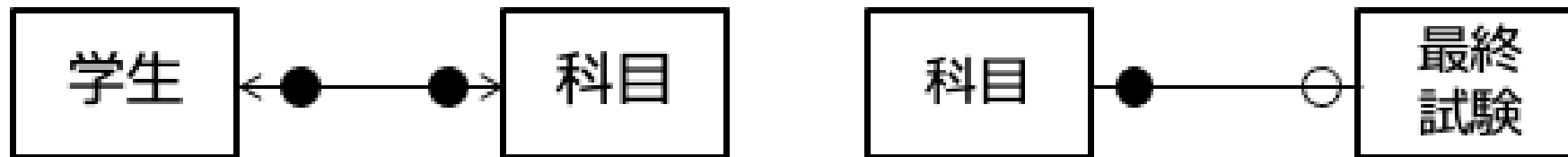
対応する現実世界のイメージ

ER図：オプションナリティ

オプションナリティとは、関連において、あるエンティティのインスタンス（実現値）が、相手側のエンティティのインスタンスに対して、**必ず存在するかどうか**を表す概念である（データベースの基礎ではやっていない）

- 多重度の最小値が0か、1以上かの違いを表しているとも言える

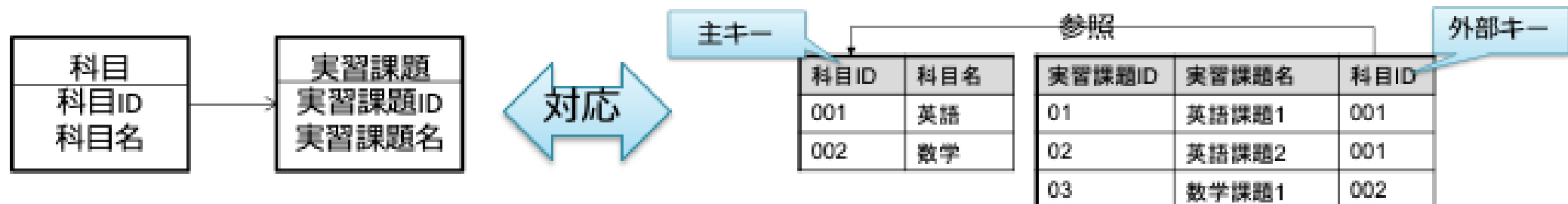
ER図では、必ず存在する場合（**必須**と呼ぶこととする）は黒丸，存在しない可能性がある場合（**オプション**と呼ぶこととする）は白丸で表す



ER図とテーブルの対応

- ER図の関連は，テーブルの主キーと外部キーの参照関係と対応する
 - ER図からテーブルを作成することができる
 - 今回は，その逆でテーブルからER図を作成する
 - しかし，対応の仕方は同じである
 - 今回の演習では主キー・外部キーを先に見つけ，それを元に関連を作成する

注：本来は外部キーとするべきだが，主キーの値ではない別の候補キーの値が設定されている場合がある．この場合も関連の存在を検討すべきだが，今回は対象外とする（例：テーブル us_states の属性 region はその可能性がある）



3. 最終課題データベースのER図作成

★注意★ ex_finalには外部キー制約がない

- 古いシステムや実際の現場のデータベースでは，外部キー制約が定義されていないことは珍しくない
- 制約が定義されていないため，外部キーはメタデータからは読み取れない．そこで今回は，**カラム名や仕様書から仮説を立て，データをSQLで調べて検証する**という調査を行う
- ネット上の情報や生成AIが答えを出してくれたとしても，**このデータベースのデータに基づく証拠**を示せなければ課題の答えにはならない

ER図における関連を作成するための調査

1. 主キーの調査

- テーブル仕様書の記述から、主キーがどれかを判断する
- データベースで定義された制約（メタデータ）をSQL（PRAGMA関数）で調べる

2. 外部キー・関連の多重度の調査

- テーブル仕様書の記述・カラム名から、外部キーの仮説を立てる
- 実際のデータをSQLで調べて仮説を検証する（外部キー制約が未定義のため、メタデータからは読めない）

3. オプションナリティの調査

- テーブル仕様書の記述から、オプションナリティを判断する
- 実際のデータが対応する個数を調査して、オプションナリティを判断する

主キーの調査（仕様書）

ordersテーブル（注文）とorder_detailsテーブル（注文の詳細）の例で説明する．

- テーブル仕様書の、テーブルの概要・カラムの意味などから主キーを判断する．必要に応じて、データを検索して確認する
- ordersテーブルのorder_idカラムは、テーブル仕様書によると「注文を識別するID」とあるから、これは主キーと考えてよい
- order_detailsテーブルは、テーブル仕様書を見ると、注文を表すorder_idとその注文に含まれる製品の情報を管理していることがわかる．1つの注文に対して、複数の製品が対応しそうだ．念のため、order_detailsテーブルのデータを見てみると、確かにorder_id 10248番には、product_id 11番・42番・72番が対応している．他のカラムは、それに従属しているから、主キーはorder_idとproduct_idの組合せである

主キーの調査（データベース）

- 主キーが定義されていれば確実．未定義の古いDBでは仕様書から判断する
- SQLiteでは，PRAGMA関数を使ってSQLで確認できる．pk列が1以上なら主キー（複合主キーの場合は主キー内の順番．0は主キーでない）

```
SELECT name, pk FROM pragma_table_info('order_details');
```

name	pk
order_id	1
product_id	2
unit_price	0

→ order_detailsの主キーは，order_idとproduct_idの複合キーとわかる（表は抜粋）

外部キー・多重度の調査（仕様書）

- あるテーブルに他のテーブルの主キーと同じ名前にカラムがある場合、それは外部キーの候補と考えられる．仕様書のカラムの意味を確認し，必要に応じてデータを検索して最終的に判断する
- 多重度は，テーブルの関連の意味から，行の対応の個数を検討し判断する
- order_detailsテーブルにあるorder_idカラムは，ordersテーブルの主キーであるから外部キーと考えて良い．よって，ordersテーブルとorder_detailsテーブルには確かに関連がある．
- この関連の多重度であるが，主キーの調査でわかったように，order_detailsテーブルは注文とその注文に含まれる製品の組を表していた．よって，一つのordersテーブルの行に対して，order_detailsテーブルの行は複数対応する．このことから，1対多であることがわかる

外部キーの調査（データベース）

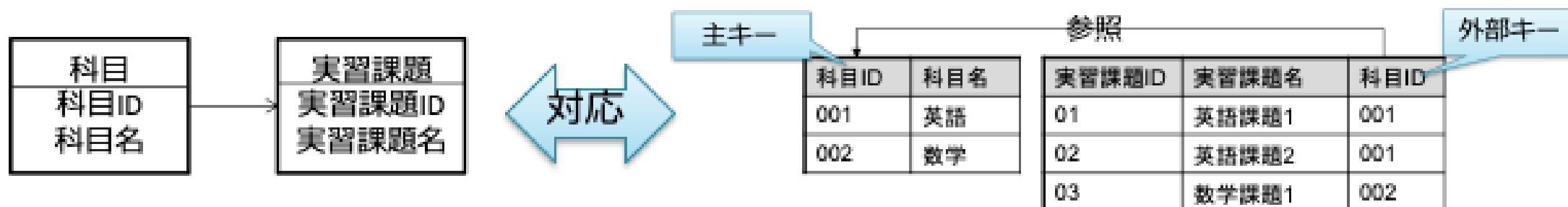
- 外部キーもデータベースに定義されていれば確実だが、**ex_finalには外部キー制約が定義されていない**。この場合は、仮説を立ててデータで検証する
 - i. カラム名・仕様書から仮説を立てる（例：order_detailsのorder_idは、ordersの主キーと同名→外部キーでは？）
 - ii. そのカラムの値がすべて参照先の主キーに存在するかをSQLで調べる

```
SELECT COUNT(*) FROM order_details
WHERE order_id NOT IN (SELECT order_id FROM orders);
```

- 実行結果が0件→すべての値が参照先に存在する→仮説が支持される
- 注意：0件は偶然の可能性もゼロではない。カラム名・仕様書の意味と合わせて総合的に判断すること

補足説明：主キー・外部キーから多重度の決定

- 主キー・外部キーの関係がわかっているときに、多重度はどうなるか？
- 下の図からわかるように、外部キーがある側が「1:多」の多の側になる（科目と実習課題は1対多）
- 注意：「1:1」となる可能性もないとは言えない（線には矢印がつかない）が、今回の最終課題では、議論を単純化するために、関連はすべて1:多とする
 - 1:1は1:多の特殊な場合と捉える
- よって、主キー・外部キーがわかれば多重度まで自動的に決まる



オプションナリティの調査（仕様書）

次にオプションナリティを調査する．仕様書からの調査について述べる．

（実際のデータを利用した調査は，次の例題1で解説する）

- オプションナリティは，まず，常識的な業務知識から判断できる場合もある．
 - 注文詳細が注文もなく存在することは，常識的に考えられない
 - よって，「注文は，注文詳細があれば必ず存在する」
→ordersテーブル側のオプションナリティは必須（黒丸）
- データの制約に関する仕様に記述があれば，それも根拠として反映させる（「同時に作成される」のような規則としての保証は，データからは得られず仕様からしか読み取れない）
 - 番号1の仕様により「注文詳細は，注文があれば必ず存在する」ことがわかる
→order_detailsテーブル側のオプションナリティは必須（黒丸）

関連作成の調査：結果のまとめ

以上の調査の結果は，配布ノートブックの記入表にまとめる．記入例：

主キー・外部キーの調査（テーブルごと）

カラム名	主キー	外部キー（参照先）	根拠（セル番号など）
order_id	○(1)	orders.order_id	仕様書・セル[A]
product_id	○(2)	products.product_id	仕様書・セル[B]

関連の調査（関連ごと）

1側テーブル	多側テーブル	1側の黒丸/白丸	多側の黒丸/白丸	根拠
orders	order_details	黒丸	黒丸	仕様書1・常識

同様の調査を行いER図を完成させる

例題1

1. territoriesテーブル（テリトリー：従業員の担当範囲）とemployee_territoriesテーブル（従業員とテリトリーの関連を表すテーブル）について、その主キー・外部キーを調査せよ
 - 主キー：pragma_table_info で調査する
 - 外部キー：カラム名から仮説を立て、SQLで検証する
2. ER図で、territoriesとemployee_territoriesの間の関連の多重度を決定せよ（どちらに矢印があるか？）
3. 上記の調査結果を、ノートブックの記入表に記録せよ（調査に使ったSQLのセルも残すこと）

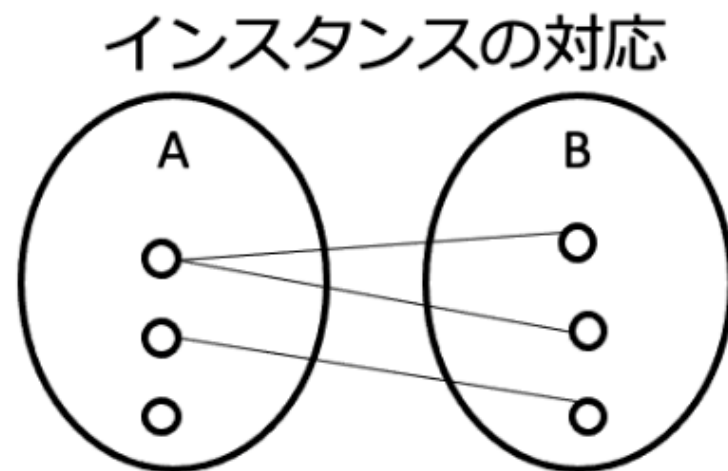
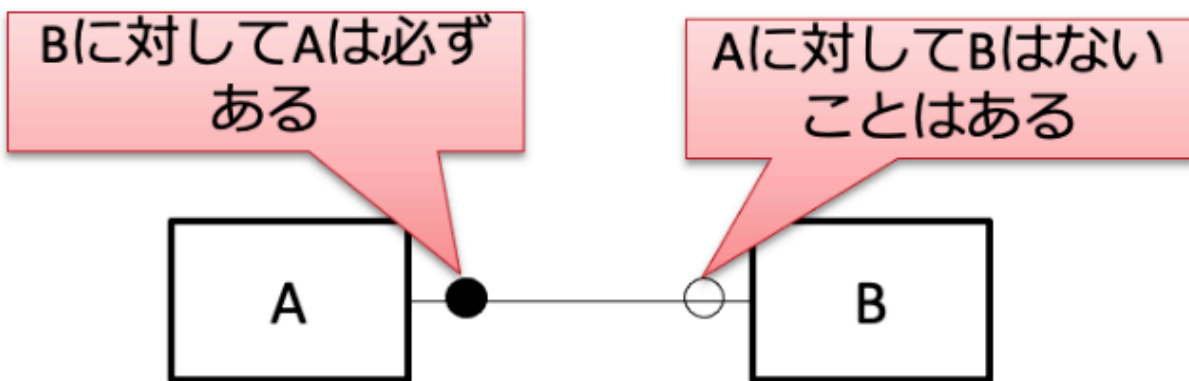
オプションナリティの調査（データベース）

- オプションナリティに関して「データの制約に関する仕様」に記載がある場合、これを元に決定する
- オプションであるかの決定は、テーブル仕様書や意味だけからは難しいときがある．このような場合は、**現在のデータから決定すること**
 - 例：外部キーの値にnullが入る→関連の相手がない→オプションとなる
 - 例：外部キーと主キーの値の種類を比較すると、外部キーの値の種類が少ない→関連の相手がいない→オプションとなる

注意：オプションナリティなどの制約はデータを入れるためのルールであり、そのルールはデータを入れる前に決まっている．よって、データからオプションナリティを求めるのは邪道とも言えるが、調査目的なので今回のやり方は許される．

参考：オプションナリティの判断の仕方

- オプションナリティをつける側のエンティティをA, 反対側をBとするときに, 「Bに対してAは必ずあるかどうか」を調べればよい
 - オプションナリティの説明をそのような記述に統一した
- わかりづらいときは, インスタンスの対応を書いてみることもヒントになる
 - 下右図でA側につながっていないインスタンスがあるということは, Bに対してAは必ずあるが, Aに対してBはないことがある状態



オプションナリティの調査例

例：employee_territoriesとterritoriesの関連のオプションナリティを現在のデータを参考にして決定せよ

1. 主キー・外部キーの確認

- この関連を表す主キー・外部キーの関係は，
 - employee_territoriesテーブルのterritory_idという外部キーが
 - territoriesテーブルのterritory_idという主キーを参照している

データによるオプションナリティの調査例

2. territories側のオプションナリティの調査

- 「territoriesはemployee_territoriesに対して必ずあるか？」を調べれば良い
- ということは、employee_territoriesのterritory_id（外部キー）にnullがあるかどうかを調べれば良い
- 以下のSQL文（territory_idがnullである行数をカウント）によって、nullがあるかを確認できる→ない→必須（黒丸）である

```
SELECT COUNT(*) FROM employee_territories WHERE territory_id IS null;
```

データによるオプションナリティの調査例

3. employee_territories側のオプションナリティの調査

- 「employee_territoriesはterritoriesに対して必ずあるか？」を調べれば良い
- ということは、employee_territoriesのterritory_idに、territoriesのすべてのterritory_idの値が存在するかを調べれば良い
- 以下のSQL文で、territoriesのterritory_idのうちemployee_territoriesに存在しない値の個数を求められる→4→0ではないのでオプション（白丸）である
- 副問合せで検索するterritory_idからnullは除外する（理由は次ページ）

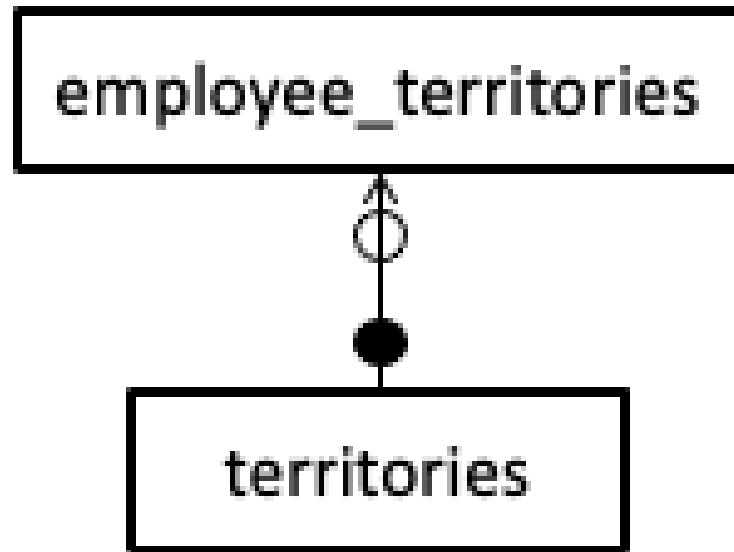
```
SELECT COUNT(territory_id) FROM territories
WHERE territory_id NOT IN (
    SELECT territory_id FROM employee_territories
    WHERE territory_id IS NOT null);
```

参考：NOT INとnull（なぜnullを除外するのか）

- SQLの真偽値はTRUE / FALSEだけでなく，第3の値「**UNKNOWN**（不明）」がある（SQL標準の3値論理）．nullが絡む比較の結果はUNKNOWNになる
- 副問合せの結果からnullが自動的に除かれることはない（残る）
- `x NOT IN (a, b, NULL)` は `x<>a AND x<>b AND x<>NULL` と等価で，`x<>NULL` はUNKNOWN．**WHEREはTRUEの行しか残さない**ため，NOT INは決してTRUEにならず必ず0件を返す
- 0件は「必須（黒丸）」という誤った結論に静かにつながるので危険
- 例：employeesのreports_to（上司ID）はnullを1件含む．部下を持たない従業員を数えると
 - 副問合せでnullを除外した場合：7人（正しい→白丸）
 - 除外しない場合：0人（黒丸と誤判定してしまう）

データによるオプションナリティの調査例

以上による結論は以下のER図となる



参考：カラムのnull有無の確認：NOT NULL制約

実はこのデータベースでは、nullを入れてはならないカラムにはNOT NULL制約（SQLでNULLを入れようとするとエラーとなる）が設定されているものもある。

employee_territoriesのterritory_idにもNOT NULL制約が設定されており、NULLを取り得ないことがわかり、前の例の2の調査の結論が得られる。主キーと同じくpragma_table_infoで調べられる（notnull列が1ならNOT NULL制約あり）

```
SELECT name, "notnull" FROM pragma_table_info('employee_territories');
```

しかし、カラムの中には、NULL不可の制約はないが、NULLは入らないデータもある（データ作成順序が縛られる等の理由）。今回については、**現在のデータを用いたSQL実行結果による判断**を優先することとする。

例題2

customersとordersの間の関連について、オプションナリティを決定せよ

最終課題 問1

以下の**5つのテーブル**について主キー・外部キーを調査し，配布ノートブックの記入表に結果を記録せよ

- 対象：orders, order_details, products, employees, employee_territories
- 主キー：pragma_table_infoで調査する（実行したセルを残すこと）
- 外部キー：カラム名・仕様書から仮説を立て，SQLで検証する（検証に使ったSQLと実行結果のセルを残すこと）
- ヒント：複合主キーのテーブル，自分自身を参照する外部キーを持つテーブルが含まれている

注意：調査対象外のテーブル（customersなど）も，関連の「1側」などとして問2には登場する（territoriesは外部キーregion_idを持つ：問3で調査する）

最終課題 問2

ex_finalデータベースの関連（customer_customer_demoとcustomer_demographicsは除く）について、多重度とオプションリティを調査し、ノートブックの記入表に記録して、ER図を完成させよ。

- 1対多の向き（どちらが多側か）と、両側のオプションリティ（黒丸/白丸）をそれぞれ記録すること
- **各行に判断の根拠を必ず記入すること**（仕様書の記述番号・常識的判断・調査SQLのいずれか）
- SQLで判断した箇所は、そのSQLと実行結果のセルがノートブックに残っていること
- ネット上の資料や生成AIを参考にすることは禁止しないが、**このデータベースのデータに基づく根拠がない解答は得点にならない**

最終課題 問3

オプションリティの調査で説明したSQLを参考にして、地域テーブルとテリトリテーブルの関連のオプションリティを調査するため、以下の2つのSQL文を作成せよ

- (1) テリトリテーブルのregion_idがnullである行数を求めるSQL文
- (2) テリトリテーブルのregion_idカラムには存在せず、地域テーブルのregion_idカラムに存在するregion_idの値の個数を求めるSQL文

最終課題 問3（続き）

従業員テーブルには、従業員の報告先（上司）を表すカラムがある（報告先従業員のIDが入る）。このカラムに関して、以下の2つのSQL文を作成せよ（このSQL文はオプションリティのヒントになっている）

(3) 報告先（上司）が存在しない従業員の人数を求めるSQL文

(4) 部下（自分を報告先としている人）を持っている従業員の人数を求めるSQL文

最終課題 問7（発展問題）：データベースの拡張設計

ex_finalに新機能を追加する．以下の要件から**1つ選び**，必要なテーブルを設計せよ

- **要件A（商品レビュー）**：顧客が製品に5段階の評価と本文のレビューを投稿できるようにしたい
- **要件B（配送状況の追跡）**：注文ごとの配送状況の変化（発送済・配達中など）を日時とともに履歴として記録したい
- **要件C（複数倉庫の在庫管理）**：製品の在庫数を複数の倉庫ごとに管理したい

解答としてノートブックに以下を記述すること

1. 追加するテーブルの定義（テーブル名・カラム・主キー・外部キー）
2. **既存テーブルとの関連**を含むER図（Mermaid記法：後述）
3. **設計理由**：主キー・外部キーの選択，多重度・オプションリティの考え方

問7の注意事項と採点基準

- **問7も採点対象**（最終レポートの配点に含む）。「発展」は難度の表示であり，任意問題ではない
- この問題には**唯一の正解はない**．採点では最終的な図の見た目よりも，**設計の理由が筋の通った言葉で説明されていること**を重視する
- 生成AIに相談してもよいが，**最初案は必ず自分で考えて書くこと**．AIの指摘を採用/不採用した場合は，その理由も記述すること
- 採点の観点
 - i. 既存テーブルへの外部キーが適切か（どのテーブルの何を参照するか）
 - ii. 主キーの選択が適切か（何が1行を識別するか）
 - iii. 多重度・オプションリティが明示され，要件と整合しているか
 - iv. 設計理由の説明に筋が通っているか

参考：Mermaid記法によるER図の記述

Mermaidはテキストでダイアグラムを記述する記法である．ER図はerDiagramで書ける．配布ノートブックのshow_er()関数で描画できる

```
erDiagram
    customers ||--o{ orders : ""
    orders ||--|{ order_details : ""
    products ||--o{ order_details : ""
```

- 1行が1つの関連を表す： 1側テーブル 記号 多側テーブル : ""
- 記号の意味は次ページの対応表を参照

参考：Mermaid記法と授業の記法の対応

授業の記法	Mermaid	意味
黒丸（必須）・1側		ちょうど1
白丸（オプション）・1側	o	0か1
黒丸（必須）・多側	{	1以上
白丸（オプション）・多側	o{	0以上

- 世の中にはER図の表記法が複数あり，Mermaidの記法（カラスの足：crow's foot 記法と呼ばれる）もその一つである
- 描画される図は，黒丸・白丸や矢印の代わりに**線の端の形**で多重度・オプションリティを表す

最終課題（第14回分）の提出方法

問1～問3・問7は、配布ノートブック（2012xxxx_DS_14.ipynb）上で解答する

- これまでの演習と同様に、WebClassの最終課題にColabの共有リンクを提出せよ
- **提出締切：8/7(金) 18:30**（問4～問6も同じ締切）
 - 間に合わなかった場合はWebClassの【締切後】欄から提出できるが、レポート点は60点以上には反映されない
- 調査に使ったSQLと実行結果のセルを残しておくこと（採点対象）
- 問4～問6（アプリケーション作成・データ分析）は第15回で説明する